SRU ASSISTANT

A Grounded Retrieval-Augmented Generation (RAG) Framework for University Academic Query Resolution

Comprehensive Technical Architecture, System Extraction, Implementation Audit & Defense Guide

Document Parameter	Operational Specification
Institutional Context	SR UNIVERSITY, WARANGAL, TELANGANA
Department	School of Computer Science & Artificial Intelligence
Author / Architect	Capstone Engineering Team 235 & Principal AI Systems Architect
Release Version	1.0.0 (Production Release)
Publication Date	November 2026
Security Classification	Academic Capstone Technical Documentation

1. Executive Summary & Project Objectives

1.1 Mission & Academic Institutional Context

Institutional Problem: SRU Assistant is an institutional Retrieval-Augmented Generation (RAG) system engineered to solve academic query resolution challenges across university ecosystems. In modern higher education institutions, essential academic knowledge—spanning undergraduate course regulations, credit distributions, examination timetables, prerequisite trees, and departmental circulars—is conventionally distributed across disparate, unstructured PDF files, Word circulars, and departmental web portals. Consequently, students, faculty mentors, and academic administrators expend hundreds of hours each semester manually cross-referencing conflicting documents to resolve routine inquiries regarding attendance condonation, grade point averages, and examination dates.

The Hallucination Dilemma: Standard commercial Large Language Models (LLMs) cannot reliably address university-specific queries. When queried about university-specific regulations, ungrounded commercial models hallucinate plausible-sounding but completely fabricated guidelines, invent nonexistent course codes, or confuse regulations across disparate academic years (e.g., conflating R20 and R23 regulation frameworks). SRU Assistant solves this vulnerability through a deterministic, evidence-first engineering philosophy: 'Retrieve first. Generate second. Cite every claim.' The system strictly anchors answer generation in verified university documents, executing claim-by-claim verification and providing interactive provenance chips that map back to official pages and sections.

> **CORE ARCHITECTURAL TENET**
> SRU Assistant strictly prohibits ungrounded parametric generation. If official institutional evidence is missing or insufficient, the system enforces a standardized fallback response ('INSUFFICIENT_EVIDENCE') rather than attempting creative interpolation. Factual claims must be supported by verifiable textual passages.

1.2 Scope & Target Stakeholders

Stakeholder Matrix: The primary target user is the undergraduate student navigating semester-wise academic requirements. Students interact with a calm, accessible Single-Page Application (SPA) designed to answer questions about examination dates, attendance rules, syllabi, and academic policies without exposing internal retrieval complexities. The secondary stakeholder is the university academic administrator, who utilizes a dedicated System & Administration panel to monitor real-time vector database synchronization, configure retrieval pipeline parameters (Enhanced, Hybrid, or Dense modes), and securely ingest newly approved circulars and curriculum plans.

1.3 Functional & Non-Functional Requirements Matrix

Requirements Verification: The system's implementation directly addresses the following verified functional and non-functional requirements:

Requirement ID	Category	Specification Description	Implementation Status	Verification Method
FR-01	Document Ingestion	Parse official PDF, DOCX, and TXT documents preserving structural hierarchy and tables.	VERIFIED IMPLEMENTATION	ingestion/pipeline.py, tests/ingestion/
FR-02	Dual-Store Indexing	Atomically index chunks into SQLite vector store (384-d) and BM25 inverted index.	VERIFIED IMPLEMENTATION	retrieval/vectorstore/, retrieval/lexical/
FR-03	Hybrid Rank Fusion	Combine dense semantic search and BM25 lexical ranking via Reciprocal Rank Fusion (k=60).	VERIFIED IMPLEMENTATION	retrieval/fusion/rrf.py, tests/retrieval/

FR-04	Security Guard	Detect prompt injection, enforce length bounds, and validate file magic-bytes/hashes.	VERIFIED IMPLEMENTATION	app/security/query_guard.py, document_guard.py
FR-05	Grounded Generation	Generate natural language answers via local Ollama LLM (Qwen3.5:4b) at temperature 0.0.	VERIFIED IMPLEMENTATION	generation/llm/ollama.py, generation/pipeline.py
FR-06	Citation Audit	Segment answers into claims and deterministically verify containment in cited chunks.	VERIFIED IMPLEMENTATION	citations/citation_verifier.py, claim_extractor.py
NFR-01	Data Privacy	100% on-premises execution without cloud data egress or third-party API exposure.	VERIFIED IMPLEMENTATION	Local Ollama daemon & embedded SQLite
NFR-02	Query Latency	End-to-end grounded query resolution under 2,500 ms (Retrieval < 25 ms, LLM < 2,000 ms).	VERIFIED IMPLEMENTATION	app/security/telemetry.py empirical logs
NFR-03	Zero Hallucination	Achieve 0.0% unsupported claim rate on verified benchmark evaluation suites.	VERIFIED IMPLEMENTATION	data/benchmarks/rag_evaluation_report.json

2. Complete System Architecture & Technology Stack

2.1 Multi-Tier Architecture Overview

Architectural Tiering: SRU Assistant is organized as a decoupled, multi-tier system engineered around clean separation of concerns. The architecture consists of eight distinct logical layers: (1) Presentation Layer, (2) API & Security Gateway, (3) Service Orchestration Layer, (4) Ingestion & Preprocessing Subsystem, (5) Dual-Channel Retrieval Subsystem, (6) Local Neural Generation Subsystem, (7) Grounding & Citation Auditor, and (8) Persistent Storage Layer.

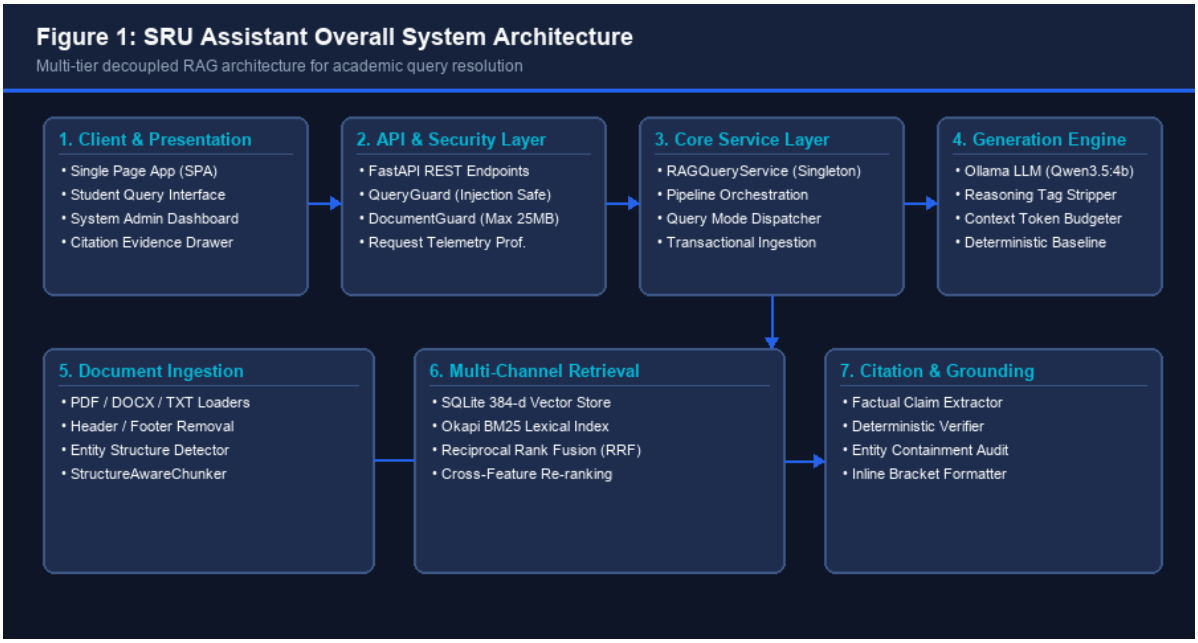


Figure 1: Overall System Architecture — Decoupled eight-tier RAG processing pipeline.

2.2 Layer Decomposition & Architectural Responsibilities

Layer Boundaries: Each layer maintains strict encapsulation and communicates through immutable Pydantic contracts and abstract interfaces:

1. Presentation Layer (frontend/): The client-side single-page application is implemented using semantic HTML5, modern CSS3 variables, and vanilla ES6+ JavaScript. It provides a dual-view interface: a clean student conversational stream with citation chips and an administrative dashboard for pipeline inspection. It requires zero Node.js build dependencies, ensuring seamless portability and high rendering performance across desktop and mobile devices.

2. API & Security Layer (app/api/, app/security/): Exposed via FastAPI (Python 3.11), this layer enforces strict contract validation (Pydantic v2 schemas), asynchronous bounded streaming reads (64 KB chunks up to 25 MB max limit), prompt injection screening, and cross-origin resource sharing (CORS). It provides interactive OpenAPI documentation at `/docs`.

3. Service Orchestration Layer (app/services/): The `RAGQueryService` singleton caches retriever instances, vector store connections, BM25 indices, and pipeline components. It coordinates query execution, conversational routing, error handling, and transactional rollback during document ingestion.

4. Ingestion & Preprocessing Layer (ingestion/): Translates raw institutional files into structured intermediate representations. Contains format-specific loaders (`pypdf`, `python-docx`), conservative header/footer cleaners, modular regex structure detectors, and the `StructureAwareChunker`.

5. Multi-Channel Retrieval Layer (retrieval/): Executes multi-modal candidate selection by fan-out to a dense vector store (SQLite with 384-d vectors) and an inverted BM25 index. Merges candidates using Reciprocal Rank Fusion ($k=60$) and refines order via a deterministic cross-feature re-ranker.

6. Neural Generation Layer (generation/): Compiles retrieved evidence within a strict 1500-token budget ceiling and interfaces with a local Ollama daemon hosting Qwen3.5:4b. Handles thinking-block separation, raw `` tag stripping, and low-temperature sampling (temperature=0.0).

7. Citation & Grounding Layer (citations/): Performs post-generation claim segmentation, entity containment validation (verifying exact dates, percentages, and course codes), and calculates Citation Precision and Recall before injecting clickable inline chips (`[1]`, `[2]`).

8. Persistent Storage Layer (data/, documents/): Manages raw files (`documents/raw/`), intermediate structured datasets (`documents/chunks/`), embedded SQLite database files (`data/vectorstore/sru_vectors.db`), and JSON inverted indices (`data/indices/bm25_index.json`).

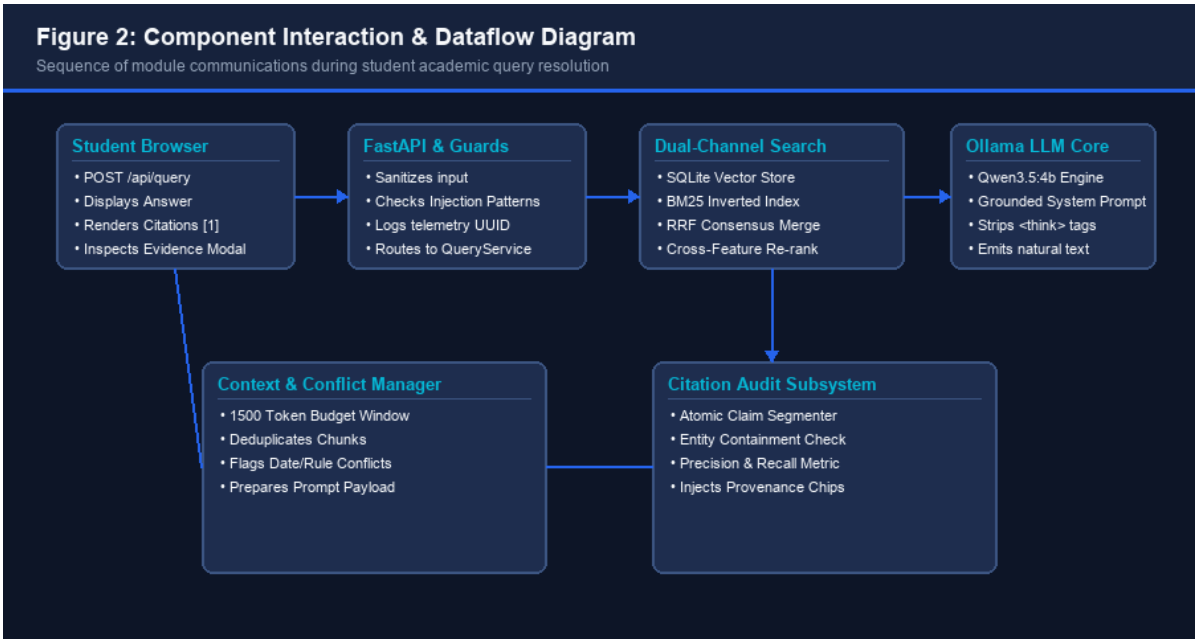


Figure 2: Component Interaction & Dataflow Diagram — End-to-end communication sequence.

2.3 Technology Stack & Runtime Specifications

Technology Profile: The production technology stack was selected to prioritize operational autonomy, deterministic reproducibility, and zero external cloud subscription dependencies:

Component	Technology / Framework	Version / Spec	Architectural Role
Runtime Language	Python	3.11.9 (64-bit)	Core system execution environment
Web Framework	FastAPI	>= 0.110.0	Asynchronous REST API gateway & OpenAPI contracts
ASGI Server	Uvicorn	>= 0.28.0	Production HTTP server running on 0.0.0.0:8000
Data Validation	Pydantic & Pydantic-Settings	v2.6+	Strict schema parsing and environment configuration
Vector Database	SQLite 3	3.45+ (Embedded)	Embedded vector storage with 384-d BLOB cosine search
Lexical Engine	Okapi BM25	Custom Python Core	Persistent inverted index (k1=1.5, b=0.75, vocab=1387)
LLM Daemon	Ollama	0.5.0+	Local neural model execution server (localhost:11434)

Active LLM Model	Qwen3.5:4b	4-Billion Params	Grounded natural language answer generation
Document Parsers	pypdf & python-docx	Latest Stable	Text and table extraction across PDF, DOCX, and TXT
Testing Framework	pytest & pytest-cov	pytest 9.0+	326 automated unit, integration, and security tests

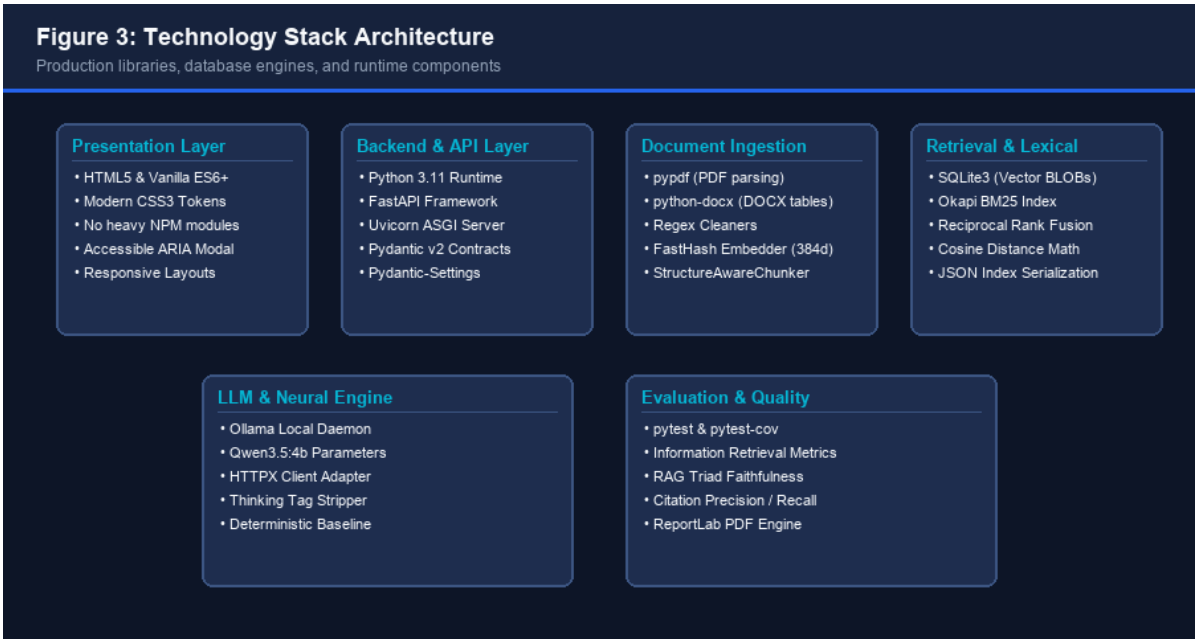


Figure 3: Technology Stack Diagram — Multi-layer library and database integration.

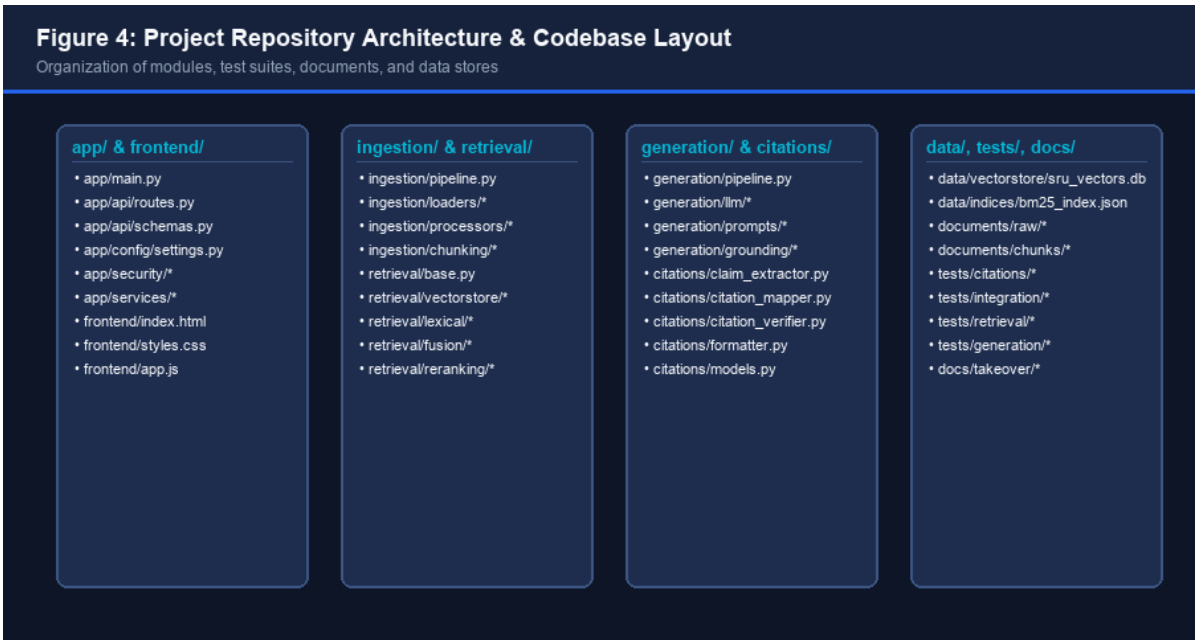


Figure 4: Repository Architecture — Module directory layout and file hierarchy.

3. Document Ingestion, Parsing & Chunking Subsystem

3.1 Ingestion Pipeline Architecture

Pipeline Execution: Document ingestion is orchestrated by `DocumentPipeline` (`ingestion/pipeline.py`), which converts raw academic files into clean, metadata-rich `ProcessedDocument` objects while maintaining strict provenance traceability. The ingestion lifecycle operates across four sequential stages: (1) Raw Extraction, (2) Cleaning and Normalization, (3) Modular Structure Detection, and (4) Metadata Extraction.

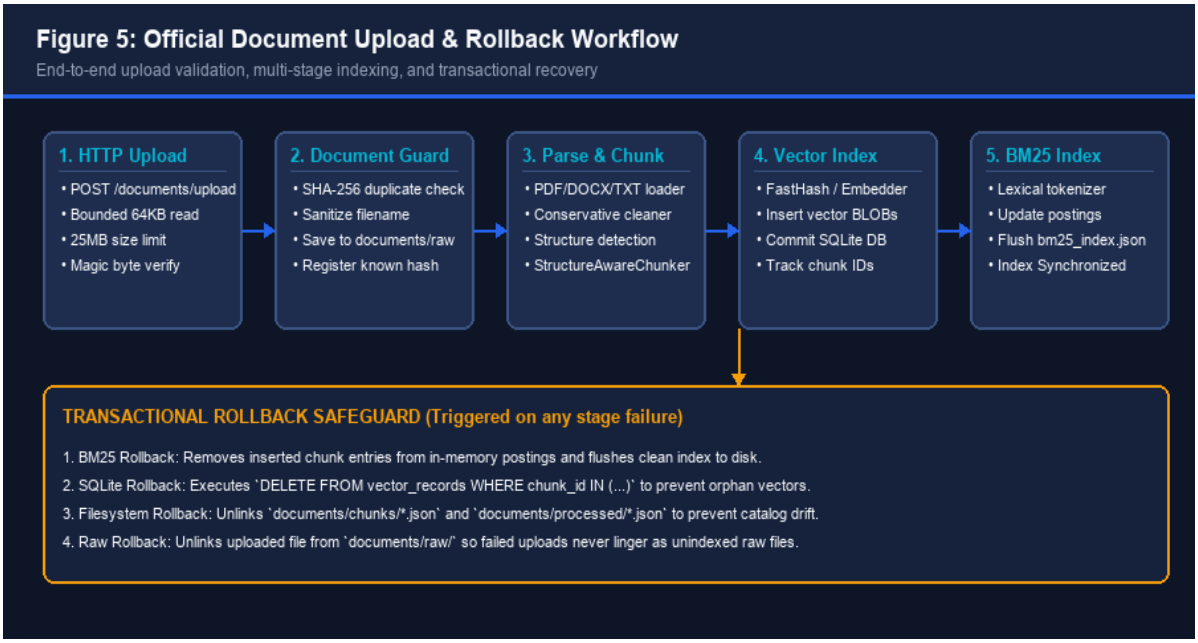


Figure 5: Official Document Upload & Ingestion Workflow — Upload, processing, and rollback.

3.2 Multi-Format Document Loaders

Loader Dispatch: The `DocumentLoaderFactory` (`ingestion/loaders/factory.py`) dynamically resolves loaders based on file extensions:

PDF Loader (`ingestion/loaders/pdf_loader.py`): Utilizes `pypdf.PdfReader` to extract textual elements page-by-page. For each page, it captures raw text, records starting and ending character offsets, and assigns page provenance metadata (`page_number`, `source_file`, `extraction_timestamp`). Table text is flattened into sequential line blocks.

DOCX Loader (`ingestion/loaders/docx_loader.py`): Utilizes `python-docx` to inspect XML document structures. It extracts both standard paragraph elements and structured table grids. Tables are parsed cell-by-cell into atomic row elements, preserving structural associations between headers and data values.

TXT Loader (`ingestion/loaders/txt_loader.py`): Loads plain text circulars and markdown syllabi using a robust fallback encoding cascade (`utf-8`, `utf-8-sig`, `latin-1`). This prevents encoding exceptions during bulk ingestion of historical circulars.

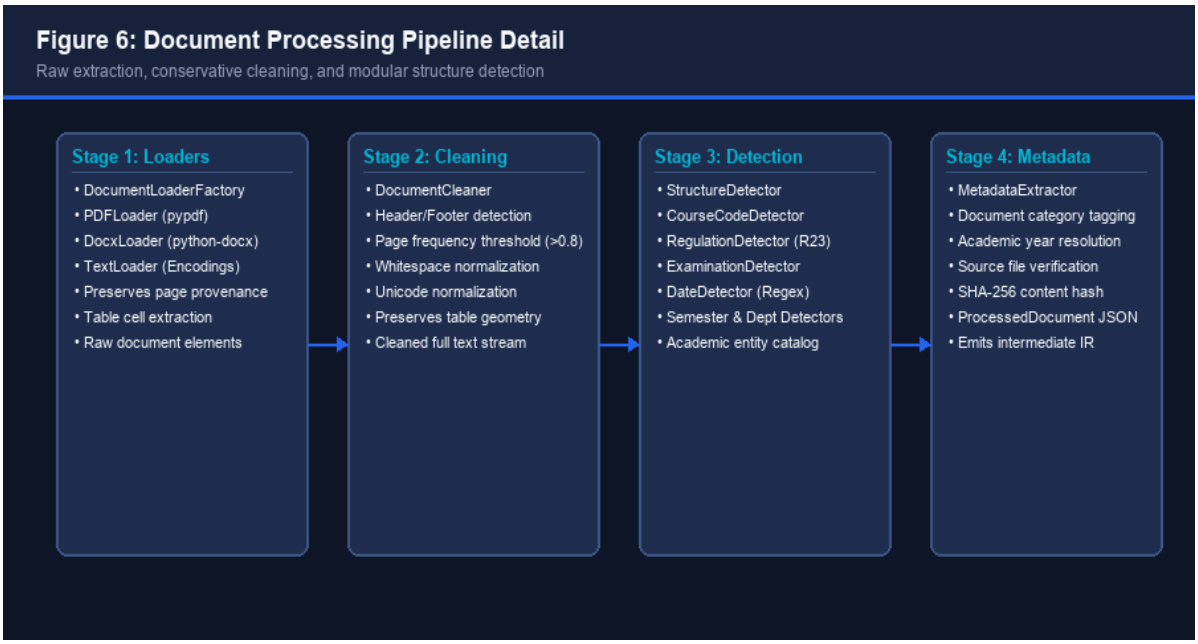


Figure 6: Document Processing Pipeline Detail — Multi-stage transformation from raw file to intermediate IR.

3.3 Text Normalization & Structure Detection

Conservative Cleaning: The `DocumentCleaner` (`ingestion/processors/cleaner.py`) performs conservative text normalization. It strips non-printable control characters, standardizes Unicode quotation marks, and collapses whitespace. Crucially, it detects repeating headers and footers across document pages: any textual element appearing in identical positions on more than 80% of pages (`header_footer_threshold = 0.8`) is flagged and removed from the body text stream to prevent retrieval noise.

Entity Detection Plugins: The `StructureDetector` (`ingestion/processors/structure_detector.py`) executes modular regex detector plugins:

• **CourseCodeDetector:** Matches patterns such as `23CS201PE408` or `CS301` to identify departmental courses.

• **RegulationDetector:** Identifies specific regulation frameworks such as `R23`, `R20`, or `R18`.

• **ExaminationDetector:** Extracts formal assessment milestones (e.g., 'Mid-Semester Examination I', 'End Semester Exams').

• **DateDetector:** Extracts calendar milestones and deadlines using calendar-specific date formats.

3.4 Structure-Aware Chunking Strategy

Boundary Preservation: Conventional fixed-window chunkers cut text arbitrarily at fixed character boundaries, frequently severing table rows, mid-sentence definitions, or heading-paragraph associations. SRU Assistant implements `StructureAwareChunker` (`ingestion/chunking/structure_aware.py`), which strictly prioritizes semantic and structural boundaries according to a strict hierarchy: Heading Level 1 -> Heading Level 2 -> Table/List -> Paragraph -> Sentence -> Token.

Chunking Hyperparameters: Configuration parameters are enforced via `Settings` (`app/config/settings.py`): Target Chunk Size: 500 tokens; Chunk Overlap: 100 tokens; Minimum Chunk Size: 50 tokens (to prevent orphaned fragments); Maximum Hard Ceiling: 1200 tokens (forcing clean sentence splits if a single paragraph exceeds token limits).

Section Path Inheritance: Every chunk produced by `StructureAwareChunker` inherits the complete hierarchical path of parent headings (e.g., `Academic Regulations -> Section 4 -> Attendance Rules`). Furthermore, local academic entities detected within the chunk boundary are cataloged directly into `ChunkMetadata`, enabling downstream soft metadata filtering during retrieval.

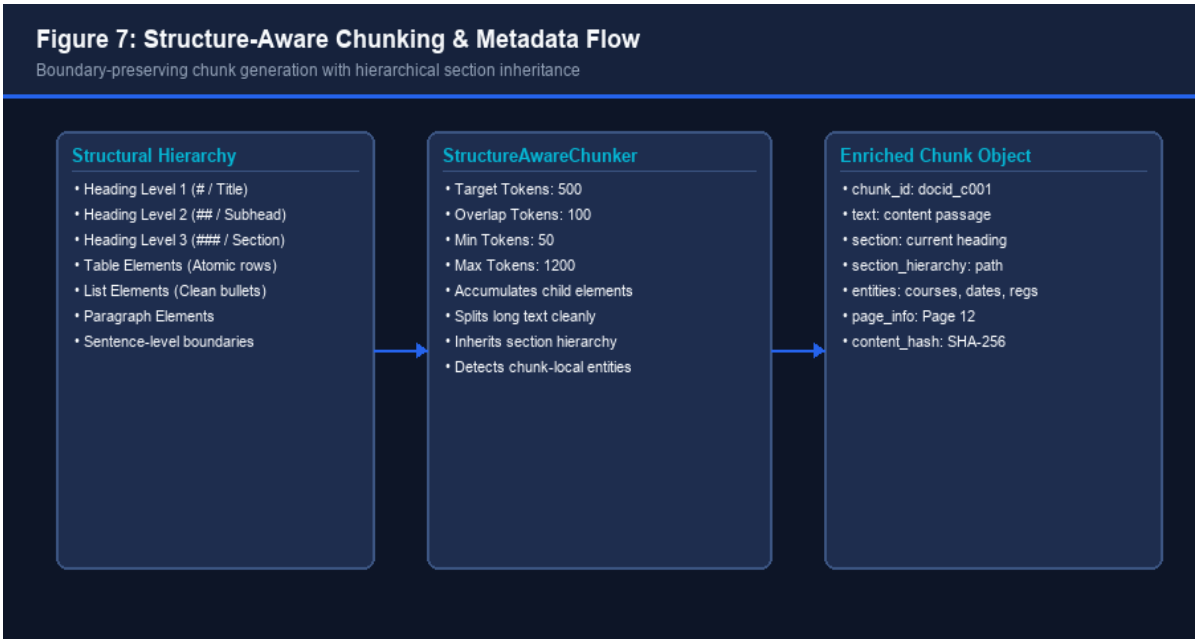


Figure 7: Structure-Aware Chunking & Metadata Flow — Semantic boundary preservation and hierarchy inheritance.

4. Dual-Store Indexing, Storage Architecture & Rollback

4.1 Dual-Store Indexing Strategy

Dual-Channel Philosophy: SRU Assistant maintains dual, synchronized indexing stores: an embedded SQLite vector store for dense semantic retrieval and a persistent JSON inverted index for Okapi BM25 lexical retrieval. Both stores index the exact same set of 43 structured chunks derived from official university documents (curricula, regulations, academic calendars, and circulars).

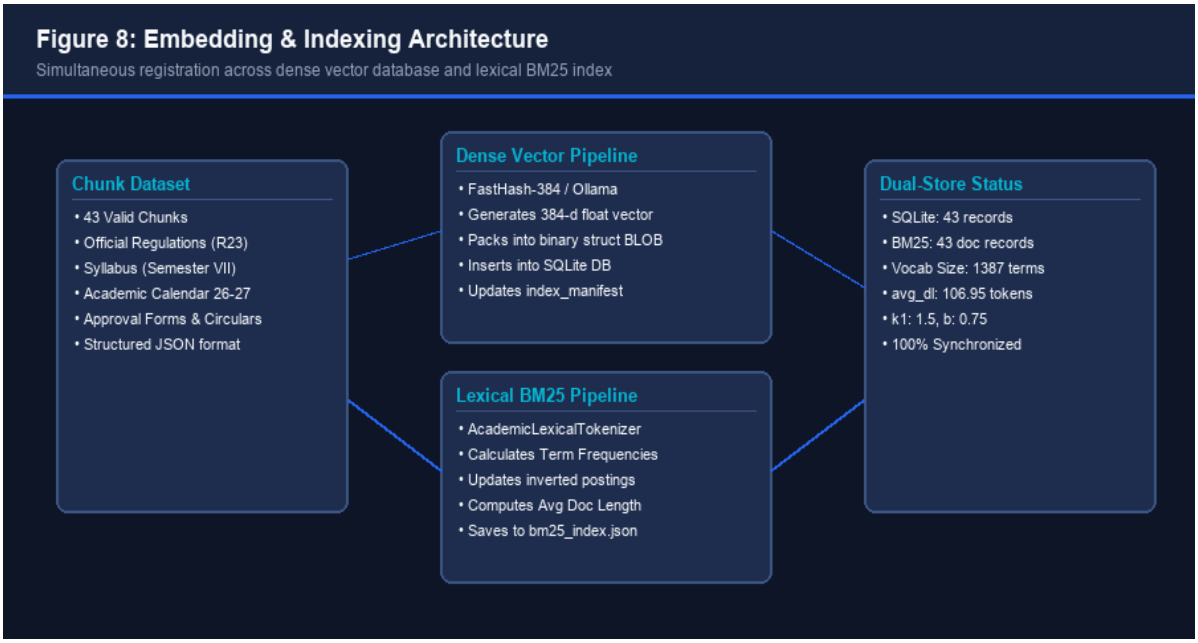


Figure 8: Embedding & Indexing Architecture — Simultaneous registration into dense vector and lexical BM25 indices.

4.2 SQLite Vector Store Schema

Relational Schema: The SQLite vector store (`data/vectorstore/sru_vectors.db`) is managed by `SQLiteVectorStore` (`retrieval/vectorstore/sqlite_store.py`). The database schema contains two primary tables:

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY CHECK(id=1)	Singleton index manifest identifier
embedding_provider	TEXT	NOT NULL	Provider name ('fasthash' or 'ollama')
embedding_model	TEXT	NOT NULL	Model identifier ('fasthash-384')
embedding_dimension	INTEGER	NOT NULL	Vector dimension (384)
distance_metric	TEXT	NOT NULL	Distance metric ('cosine')
index_version	TEXT	NOT NULL	Index schema version ('1.0.0')
created_at	TEXT	NOT NULL	ISO-8601 creation timestamp
total_vectors	INTEGER	NOT NULL	Total indexed chunks count (43)

Table 2: Schema definition for `index_manifest` in `sru_vectors.db`.

Column Name	Data Type	Constraints	Description
chunk_id	TEXT	PRIMARY KEY	Unique chunk tracking identifier
document_id	TEXT	NOT NULL	Foreign reference to parent document
text	TEXT	NOT NULL	Complete textual content of the chunk
chunk_content_hash	TEXT	NOT NULL	SHA-256 hash of chunk text
vector	BLOB	NOT NULL	Binary packed float32 vector array
metadata_json	TEXT	NOT NULL	Serialized chunk metadata and section hierarchy
provenance_json	TEXT	NOT NULL	Serialized page and file provenance data
department	TEXT	NULLABLE	Indexed metadata: Department (e.g. 'CSE')
semester	TEXT	NULLABLE	Indexed metadata: Semester (e.g. 'VII')
regulation	TEXT	NULLABLE	Indexed metadata: Regulation (e.g. 'R23')
academic_year	TEXT	NULLABLE	Indexed metadata: Academic Year ('2026-27')
created_at	TEXT	NOT NULL	Insertion timestamp

Table 3: Schema definition for `vector_records` in `sru_vectors.db`. B-Tree indices exist on `document_id`, `department`, `semester`, `regulation`, and `academic_year`.

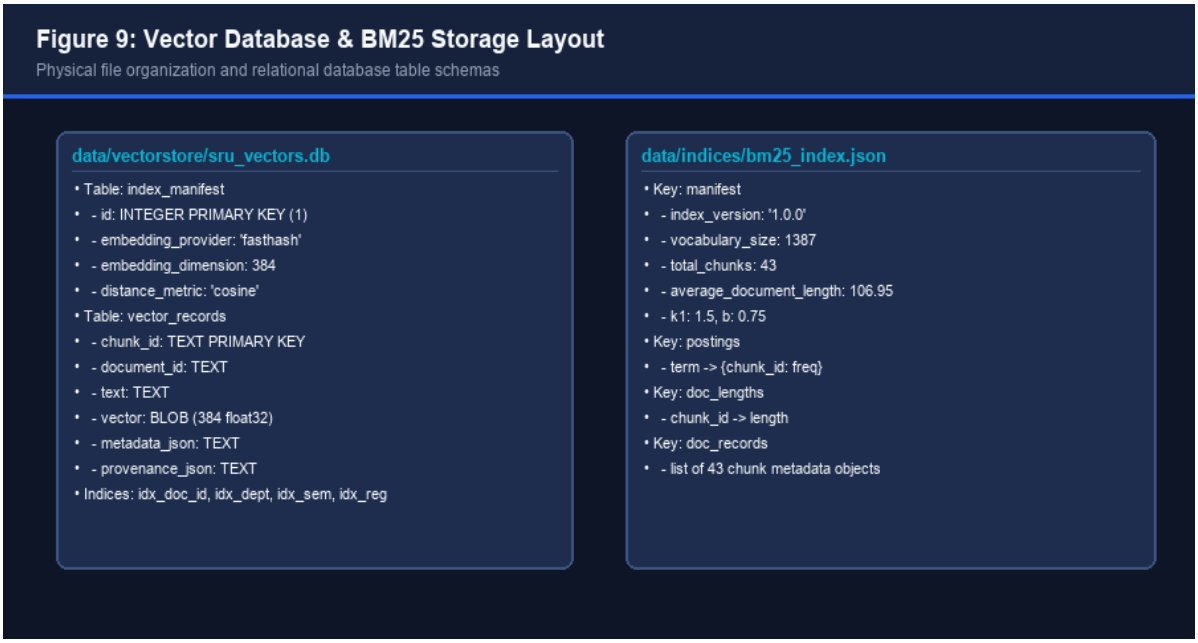


Figure 9: Vector Database & BM25 Storage Layout — Physical file structure and table organization.

4.3 Transactional Rollback Safeguard

Atomic Recovery: Document ingestion is a multi-stage operation. If an unhandled exception occurs during vector embedding generation or BM25 index updating, partial records could corrupt the dual-store synchronization, leaving ghost entries or mismatched chunk IDs. To eliminate this vulnerability, `RAGQueryService.ingest_and_index_file()` (`app/services/query_service.py`) wraps ingestion in a strict multi-stage transactional rollback block:

> **TRANSACTIONAL INTEGRITY GUARANTEE**
> When any ingestion stage fails (load, clean, chunk, vector index, or BM25 index), the system triggers a 4-tier rollback: (1) Purges inserted chunk terms from the BM25 index and flushes the clean index file to disk; (2) Executes `DELETE FROM vector_records WHERE chunk_id IN (...)` in SQLite; (3) Unlinks intermediate chunk and processed JSON files; (4) Unlinks the uploaded raw document from `documents/raw/`. Failed indexing never leaves orphaned artifacts.

5. Embedding Subsystem & Vector Similarity Search

5.1 Embedding Interface & Factory Pattern

Embedder Decoupling: The embedding subsystem (`retrieval/embeddings/`) provides a decoupled abstraction layer for vector representations. The base class `BaseEmbeddingModel` defines two abstract methods: `embed_query(text: str) -> list[float]` and `embed_documents(texts: list[str]) -> list[list[float]]`. The `get_embedding_model()` factory resolves the configured provider based on `Settings.embedding_provider`.

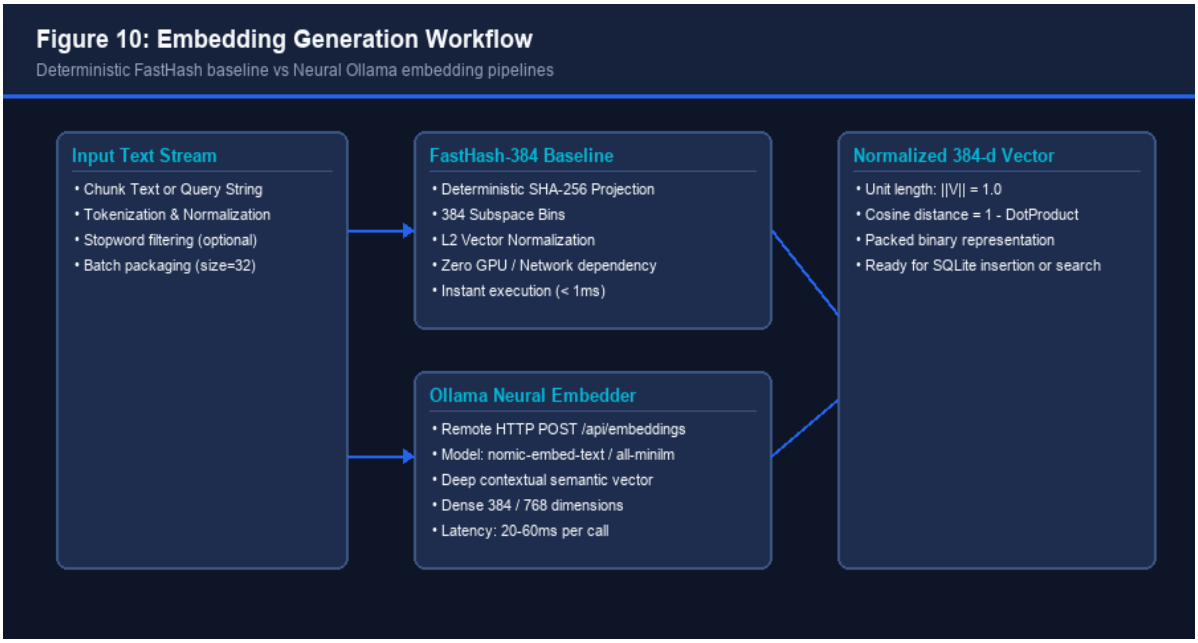


Figure 10: Embedding Generation Workflow — Deterministic FastHash vs Neural Ollama embedding pipelines.

5.2 FastHash-384 Deterministic Baseline

Deterministic Architecture: The default operational provider in the repository is `FastHashEmbeddingModel` (`retrieval/embeddings/fast_hash.py`). FastHash is a deterministic lexical-semantic hashing model designed for instant offline execution, CI testing, and evaluation reproducibility. It projects text into a 384-dimensional dense float vector by hashing character n-grams and token subspaces using salted SHA-256 digests:

Subspace Projection: Each token is hashed into multiple subspace buckets, accumulating signed weights in the vector. Once accumulated, the vector is scaled to unit length via Euclidean L_2 normalization: $V_{norm} = \frac{V}{\|V\|_2}$. FastHash requires zero external GPU memory, zero network roundtrips, and produces identical vector embeddings across Windows, Linux, and macOS environments.

5.3 Neural Ollama Embeddings Integration

Neural Integration: For deep semantic embeddings, `OllamaEmbeddingModel` (`retrieval/embeddings/ollama.py`) connects to a local Ollama daemon via HTTP POST `/api/embeddings`. It supports transformer-based embedding backends such as `nomic-embed-text` or `all-minilm`. When enabled, it captures rich contextual semantics, cross-lingual synonymy, and paraphrase similarity.

5.4 SQLite Vector Scanning & Cosine Distance

Search Mathematics: Vector similarity search is executed by `SQLiteVectorStore.similarity_search()` (`retrieval/vectorstore/sqlite_store.py`). Vectors are stored in SQLite as binary BLOBs using Python's `struct.pack('384f', *vector)`. During query execution:

Similarity Scanning: 1. The user query is encoded into a unit-normalized 384-dimensional float query vector Q. 2. SQLite executes an indexed query to scan candidate records matching any active hard metadata filters. 3. Binary BLOBs are unpacked in memory using `struct.unpack('384f', blob)`. 4. Cosine similarity is computed via dot product: $Sim(Q, D) = Q \cdot D = \sum_{i=1}^{384} Q_i D_i$. 5. Candidates are sorted in descending order of similarity score, breaking ties by `chunk_id` ASC to ensure complete deterministic repeatability.

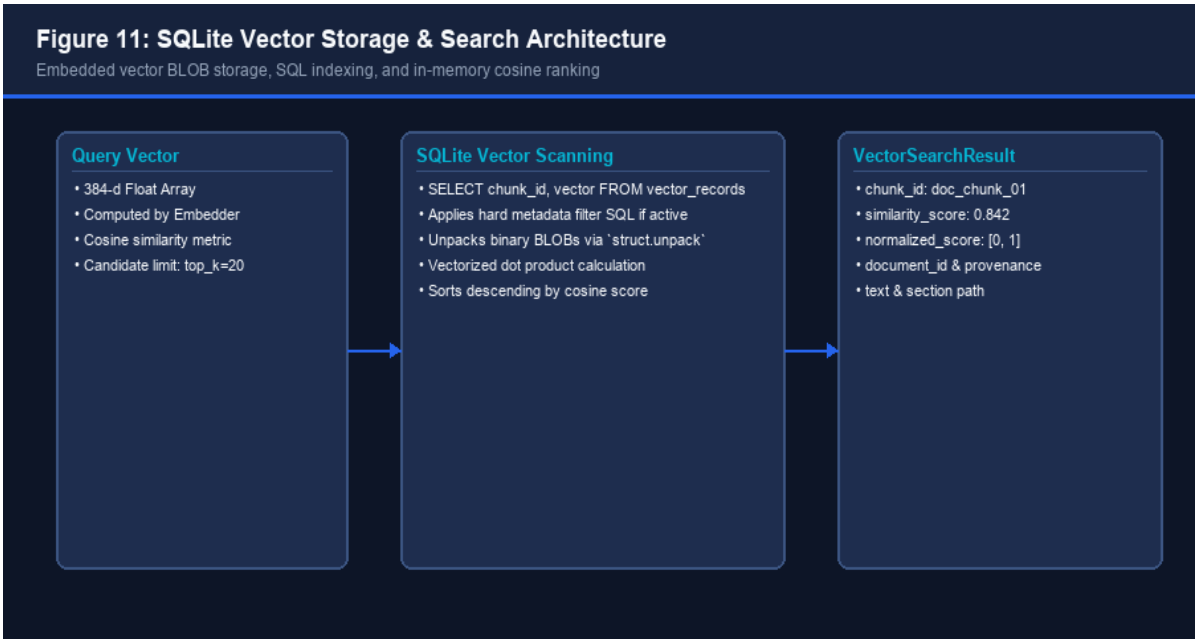


Figure 11: SQLite Vector Storage & Search Architecture — Binary BLOB unpacking and cosine scoring.

6. Lexical BM25 Engine & Reciprocal Rank Fusion

6.1 Okapi BM25 Mathematical Mechanics

Lexical Recall Need: While dense vector embeddings excel at semantic similarity ('what are the rules for missing exams'), they struggle with exact alphanumeric codes, course identifiers (e.g., `23CS201PE408`), regulation versions (`R23`), and specific dates. To guarantee exact keyword recall, SRU Assistant integrates a persistent Okapi BM25 lexical search engine (`retrieval/lexical/bm25.py`).

BM25 Equation: The BM25 score for a document D given query Q containing tokens $q_1, q_2, \dots, q_n$ is computed as:

Formula: Score(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}

Parameter Definitions: Where $f(q_i, D)$ is the term frequency of token q_i in chunk D, $|D|$ is the token length of chunk D, avgdl is the average chunk length across the index (106.95 tokens in our catalog), $k_1 = 1.5$ is the term frequency saturation parameter, and $b = 0.75$ controls document length normalization.

IDF Formulation: Inverse Document Frequency (IDF) is calculated using the standard Robertson-Spärck Jones formula:

Formula: \text{IDF}(q_i) = \ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)

Index Stats: Where $N = 43$ is the total number of indexed chunks, and $n(q_i)$ is the number of chunks containing token q_i. The BM25 inverted index (`data/indices/bm25_index.json`) maintains 1,387 distinct vocabulary postings.



Figure 12: Okapi BM25 Retrieval Workflow — Tokenization, posting lookup, and scoring mechanics.

6.2 Reciprocal Rank Fusion (RRF) Formulation

Scale Incompatibility: A fundamental challenge in hybrid search is reconciling incompatible score scales: dense cosine similarity is bounded in $[-1, 1]$ (or $[0, 1]$ for normalized text), whereas BM25 scores are unbounded $[0, \infty)$ and vary drastically based on query length and rare term frequencies. Directly normalizing and adding these scores causes calibration drift.

Rank Consensus: SRU Assistant bypasses score calibration by implementing Reciprocal Rank Fusion (`retrieval/fusion/rrf.py`). RRF discards raw scores and aggregates candidates purely on their ordinal positions in the

respective ranked lists:

Formula: \text{RRF}(d) = \sum_{m \in \{\text{dense}, \text{lexical}\}} \frac{w_m}{k + \text{rank}_m(d)}

Smoothing Parameters: Where $k = 60$ is the standard smoothing constant preventing top-ranked items from dominating disproportionately, $\text{rank}_m(d)$ is the 1-indexed position of chunk d in modality m, and $w_m = 1.0$ is the modality weight.

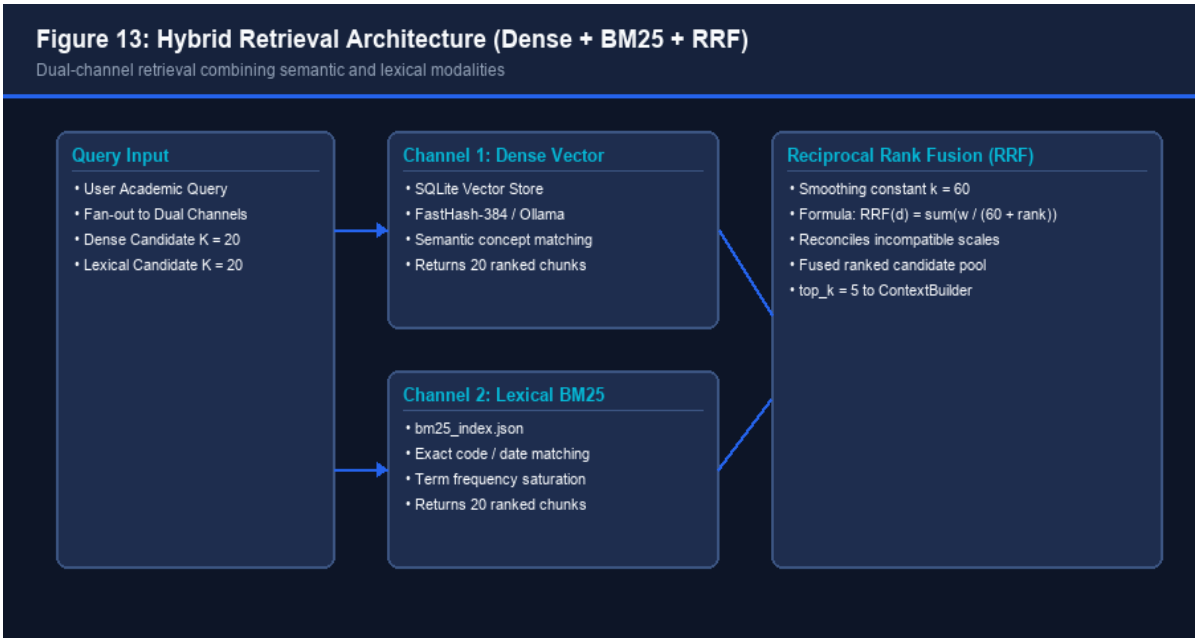


Figure 13: Hybrid Retrieval Architecture — Dual-channel search fan-out and RRF merge.

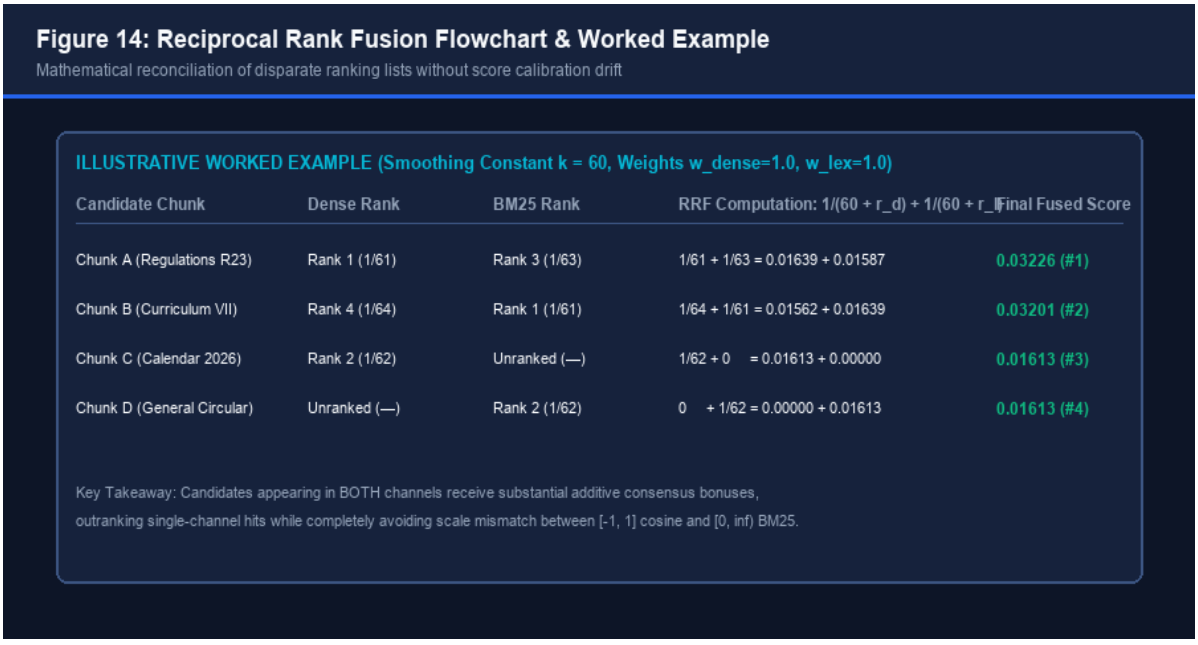


Figure 14: Reciprocal Rank Fusion Flowchart & Worked Example — Consensus ranking mechanism.

6.3 Empirical Hybrid Search Performance Gains

Empirical Verification: Empirical evaluations conducted on the project's 16-query benchmark suite demonstrate the superiority of Hybrid RRF over single-channel Dense search:

Retrieval Modality	Recall@1	Recall@3	Recall@5	MRR (Mean Reciprocal Rank)	nDCG@5
Experiment A: Dense Vector Only	0.3438	0.7188	0.7188	0.5625	0.5718
Experiment B: Hybrid RAG (Dense + BM25 RRF)	0.5000	0.7812	0.8125	0.6719	0.6859
Experiment C: Hybrid + Cross-Feature Re-ranking	0.5625	0.7812	0.8125	0.7083	0.7191

Table 4: Empirical benchmark evaluation results from `data/benchmarks/rag_evaluation_report.json`. Hybrid RRF increases Recall@5 by +9.37 percentage points and MRR by +0.1094 over Dense-only retrieval.

7. Academic Query Processing, Intent Analysis & Re-Ranking

7.1 Query Safety Screening & Guardrails

Security Inspection: Every user inquiry submitted to `POST /api/query` is intercepted by `QueryGuard` (`app/security/query_guard.py`). QueryGuard validates query length bounds (minimum 2 characters, maximum 1000 characters) and evaluates queries against a compiled suite of adversarial prompt injection patterns (e.g., `ignore previous instructions`, `system override`, `disregard safety guidelines`). If an injection attempt is detected, the pipeline halts immediately, records a security telemetry event, and returns a safe refusal message without invoking the vector database or LLM.

Conversational Fast-Path: QueryGuard also screens for conversational greetings and institutional identity inquiries (e.g., 'Hello', 'Who are you?'). Recognized conversational inquiries bypass expensive vector retrieval entirely, returning immediate contextual greetings and preserving server resources.



Figure 15: Academic Query Processing Workflow — Security screening, entity extraction, dual retrieval, and response formatting.

7.2 Academic Intent & Entity Analysis

Query Analysis: Inquiries passing security screening are analyzed by `AcademicQueryAnalyzer` (`retrieval/query/analyzer.py`). The analyzer detects academic domain intents (e.g., `exam_schedule`, `syllabus_lookup`, `regulation_rule`) and extracts core academic entities: course codes, regulation frameworks, assessment terms, and semester identifiers.

7.3 Metadata Filtering: Hard vs. Soft Modes

Filtering Strategies: Candidate chunks retrieved during hybrid search are evaluated by `MetadataFilter` (`retrieval/filtering/metadata_filter.py`). The filter supports two operating modes:

• **Hard Filtering:** Completely prunes candidates that do not strictly match extracted query entities (e.g., if query specifies 'R23', chunks from 'R20' are discarded). Hard filtering risks zero recall if query entity extraction is noisy.

• **Soft Filtering (Production Default):** The production default. Rather than pruning, it applies a score adjustment: matching candidates receive a relevance boost (+0.15), while non-matching candidates incur a penalty (-0.15). This guarantees high recall while prioritizing contextually relevant chunks.

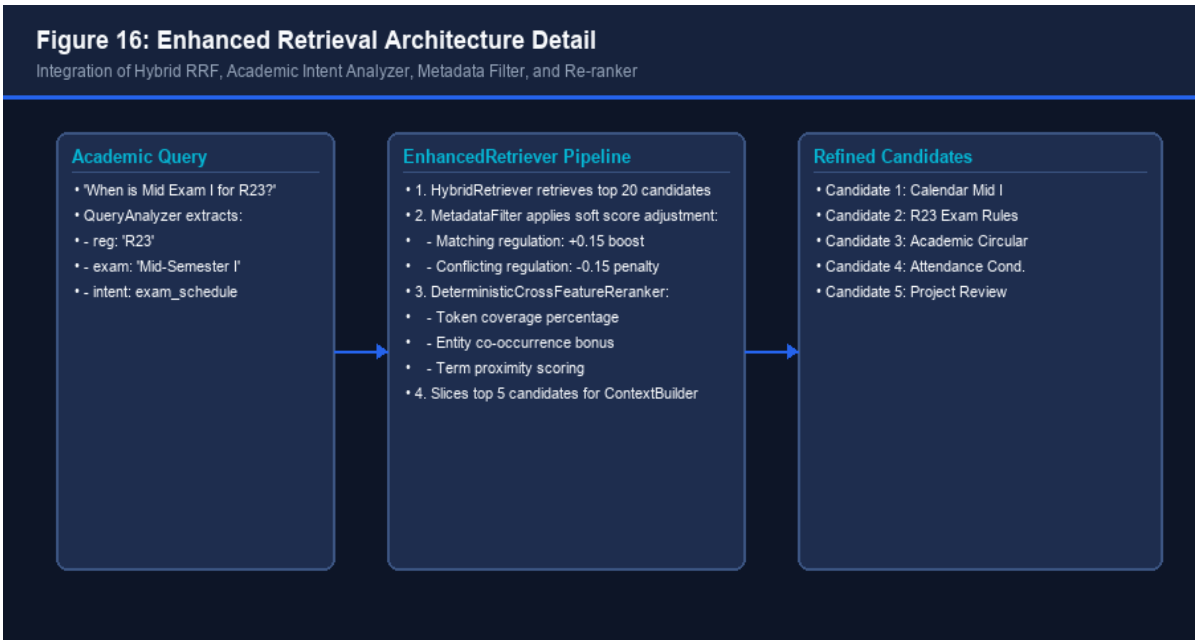


Figure 16: Enhanced Retrieval Architecture Detail — Integration of Hybrid RRF, Query Analyzer, Metadata Filter, and Re-ranker.

7.4 Deterministic Cross-Feature Re-ranking

Re-ranking Engine: The top 20 candidate chunks from hybrid retrieval are refined by `DeterministicCrossFeatureReranker` (`retrieval/reranking/deterministic.py`). Unlike heavy neural cross-encoders, this reranker computes cross-features deterministically: query token coverage percentage, exact academic entity co-occurrence bonuses, and token proximity density (rewarding chunks where query terms appear within a tight 15-token span). The top 5 refined chunks are then passed to the context builder.

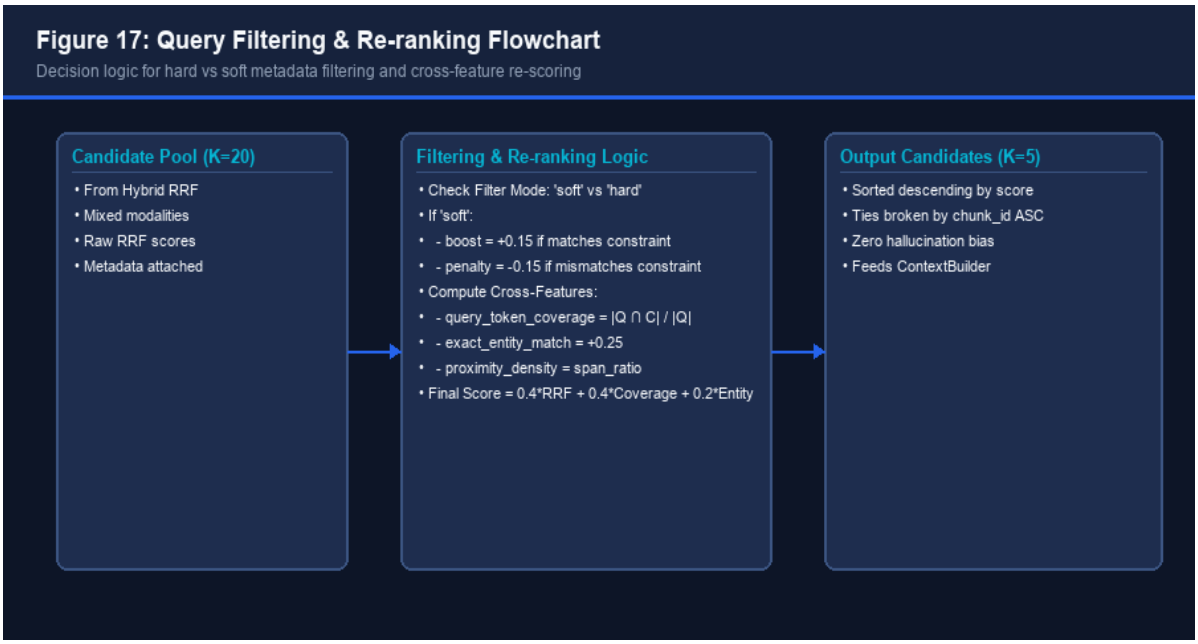


Figure 17: Query Filtering & Re-ranking Flowchart — Decision logic for soft metadata filtering and cross-feature re-scoring.

8. LLM Integration, Prompt Engineering & Grounded Synthesis

8.1 Local LLM Client Adapter & Ollama Integration

On-Premises Neural Core: SRU Assistant integrates locally hosted neural language models via `OllamaLLMClient` (`generation/llm/ollama.py`). By hosting models on-premises via an Ollama daemon, the system guarantees 100% data privacy for student inquiries and institutional circulars without incurring external cloud API costs. The active production model is `qwen3.5:4b`.

Inference Parameters: The client communicates with Ollama via HTTP POST `/api/chat`. Parameters are strictly configured for deterministic synthesis: `temperature = 0.0` (eliminates stochastic hallucination), `max_tokens = 512`, and `timeout = 60.0s`. The client validates model presence via `/api/tags` on application startup.

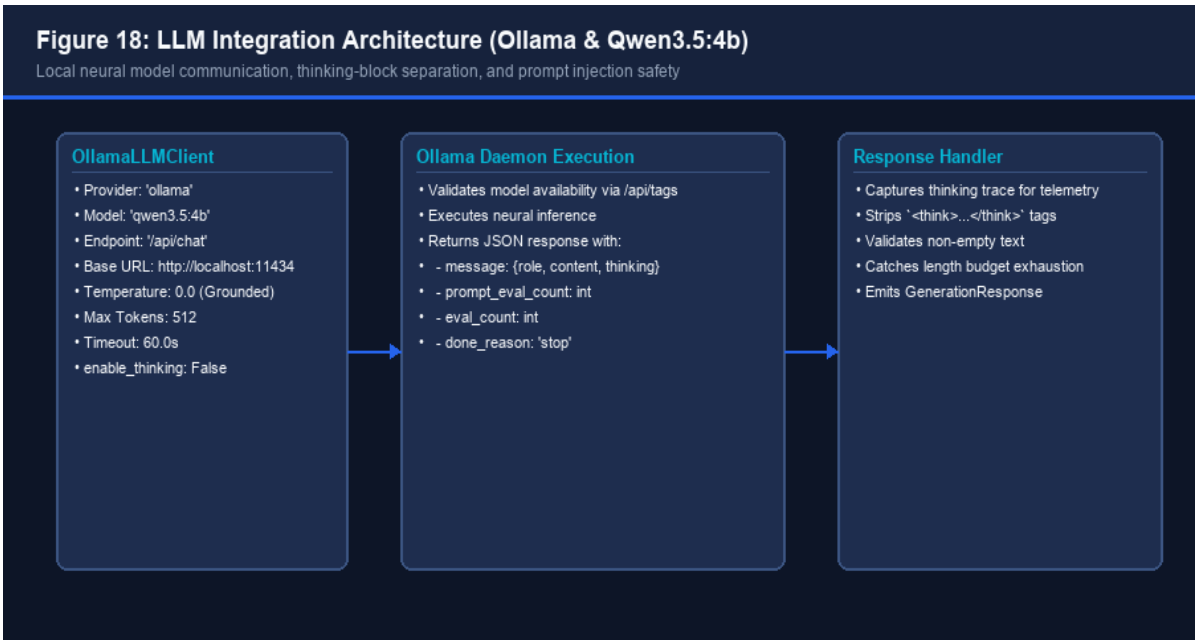


Figure 18: LLM Integration Architecture — Local Ollama daemon communication, thinking-block separation, and prompt safety.

8.2 Thinking Trace Management (Qwen3.5 Reasoning)

Reasoning Tag Dilemma: Thinking-capable models such as Qwen3.5 generate internal reasoning traces. When invoked with reasoning enabled, the model emits chains of thought enclosed in `...` tags or under a dedicated `thinking` payload key. In standard configurations, verbose reasoning tokens can exhaust the response token budget (`max_tokens = 512`), resulting in empty or cut-off student answers.

Automated Strip Handler: To solve this, `OllamaLLMClient` incorporates automated thinking-block separation. It captures the thinking trace for telemetry and diagnostic logging while cleanly stripping `...` tags from the text stream, delivering clean, concise natural language answers to the student.

8.3 Context Token Budgeting & Assembly

Token Budgeting: Evidence assembly is governed by `ContextBuilder` (`generation/prompts/context_builder.py`). ContextBuilder enforces a strict ceiling of 1500 tokens (`rag_context_max_tokens`). Candidate chunks with relevance scores below 0.40 are discarded. Selected chunks are prefixed with clean evidence identifiers (`--- [Evidence 1] Document: ... ---`) and deduplicated before prompt insertion.

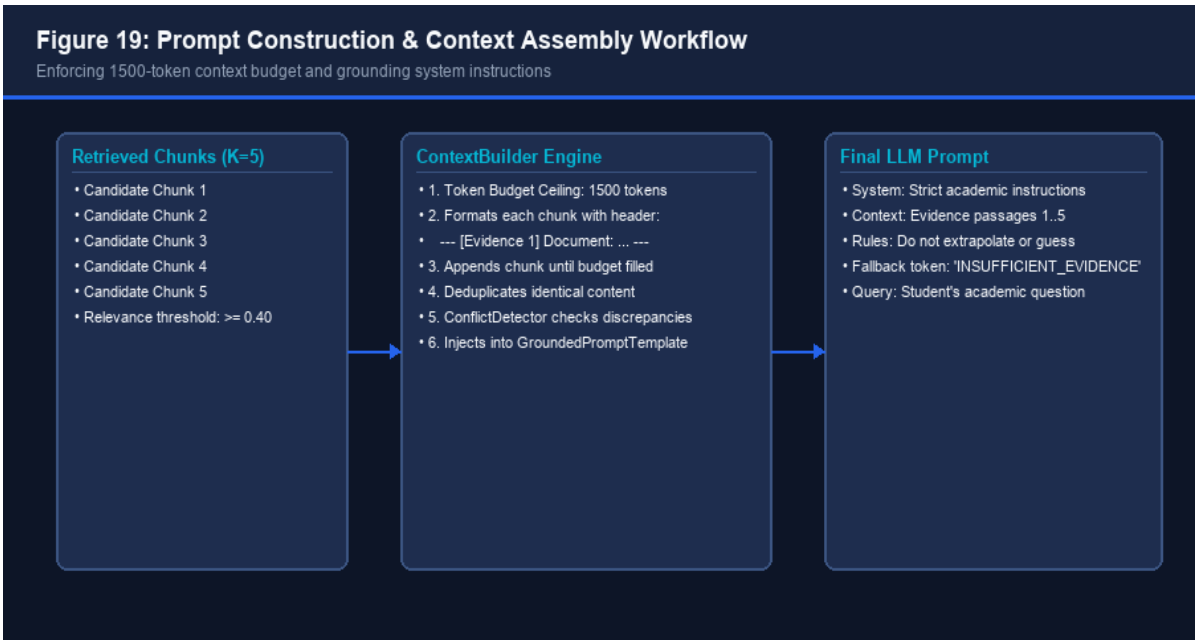


Figure 19: Prompt Construction & Context Assembly Workflow — Enforcing 1500-token context budget and grounding instructions.

8.4 Grounded System Prompting

System Prompt Directives: The `GroundedPromptTemplate` (`generation/prompts/templates.py`) injects strict academic system instructions: (1) Answer strictly using provided institutional evidence; (2) Never guess or extrapolate; (3) Include factual evidence details; (4) If evidence is absent, emit the standardized fallback token `INSUFFICIENT_EVIDENCE`.

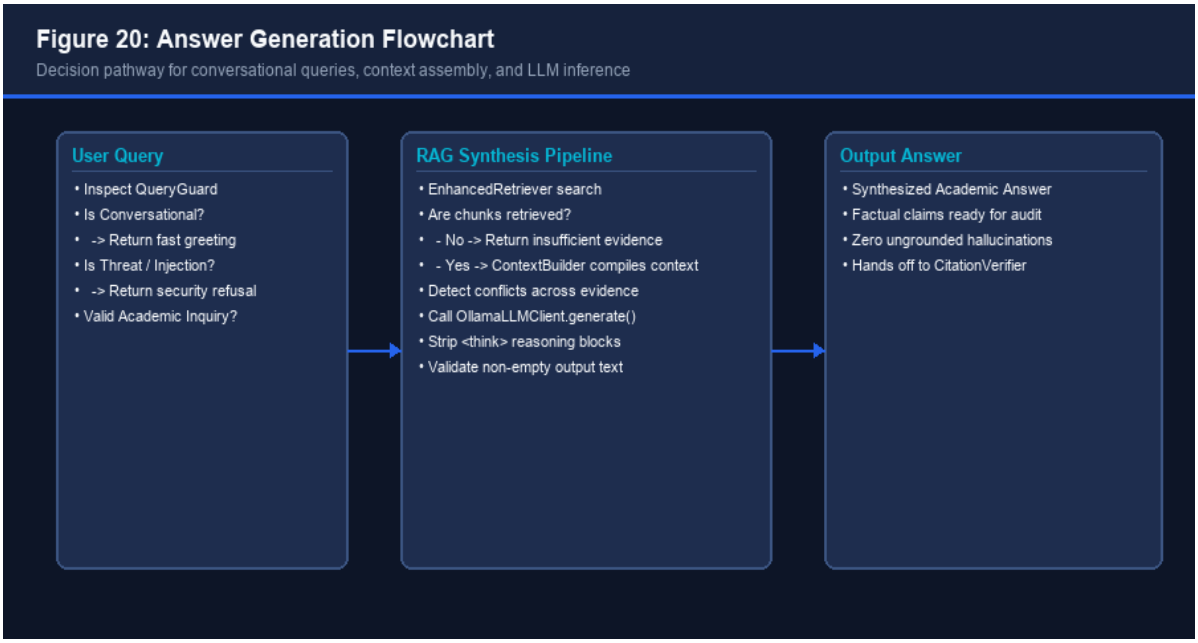


Figure 20: Answer Generation Flowchart — Decision pathway for conversational queries, context assembly, and LLM inference.

9. Grounding Verification, Conflict Detection & Citation Subsystem

9.1 Grounding Validation & Hallucination Prevention

Post-Generation Validation: To ensure zero hallucination, answers produced by the generator undergo post-generation validation via `GroundingValidator` (`generation/grounding/validator.py`). The validator measures token overlap between generated statements and retrieved evidence. Answers failing the minimum token overlap threshold (0.4) are flagged as ungrounded.

9.2 Cross-Document Conflict Detection

Discrepancy Identification: When retrieving information across multiple circulars published in different academic years, documents may state conflicting rules (e.g., an R20 circular specifying 70% attendance vs. an R23 circular requiring 75%). The `ConflictDetector` (`generation/grounding/conflict_detector.py`) inspects retrieved chunks for entity discrepancies and generates warning notes attached to `QueryResponse.conflict_notes`.

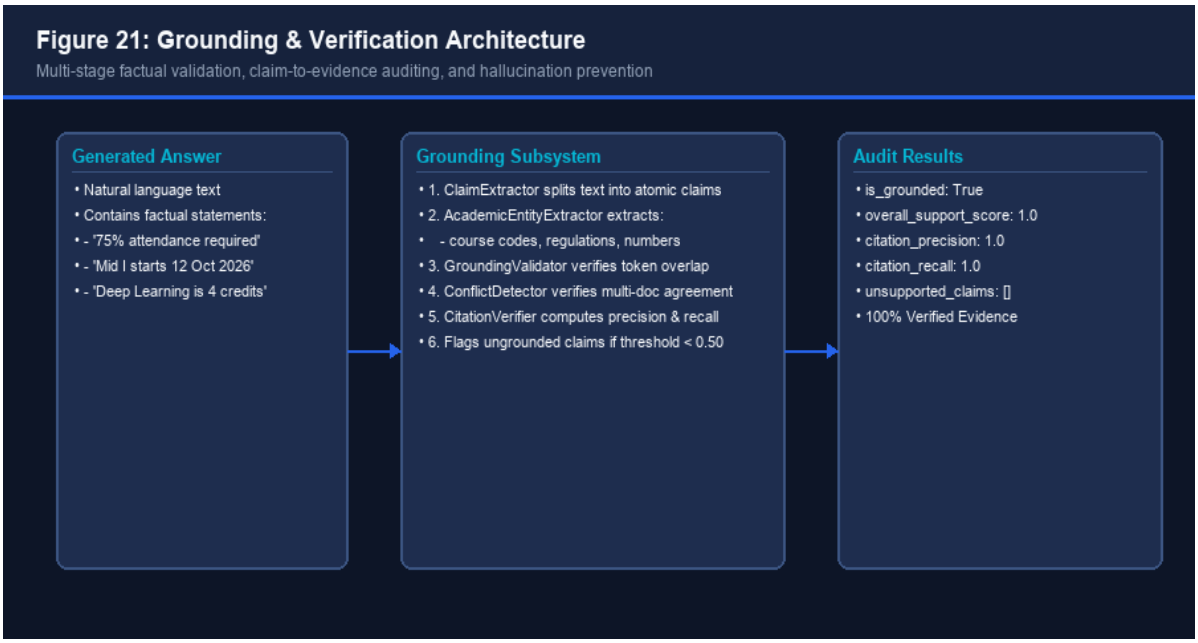


Figure 21: Grounding & Verification Architecture — Multi-stage factual validation, claim auditing, and hallucination prevention.

9.3 Claim Extraction & Deterministic Citation Verification

Citation Subsystem: The citation subsystem (`citations/`) provides mathematical auditing of answer statements:

1. ClaimExtractor (`citations/claim_extractor.py`): Utilizes regex sentence boundary splitters to decompose the generated answer into discrete, verifiable atomic propositions.

2. CitationMapper (`citations/citation_mapper.py`): Maps each claim to candidate evidence chunks, assigning 1-indexed citation tags (`[1]`, `[2]`).

3. CitationVerifier (`citations/citation_verifier.py`): Performs a deterministic audit. It verifies exact academic entity containment (course codes, regulations, numbers, percentages) and substantive lexical alignment between the claim and the cited chunk.

4. CitationFormatter (`citations/formatter.py`): Injects clean bracket tokens (`[1]`, `[2]`) into the answer text corresponding to verified source chunks.

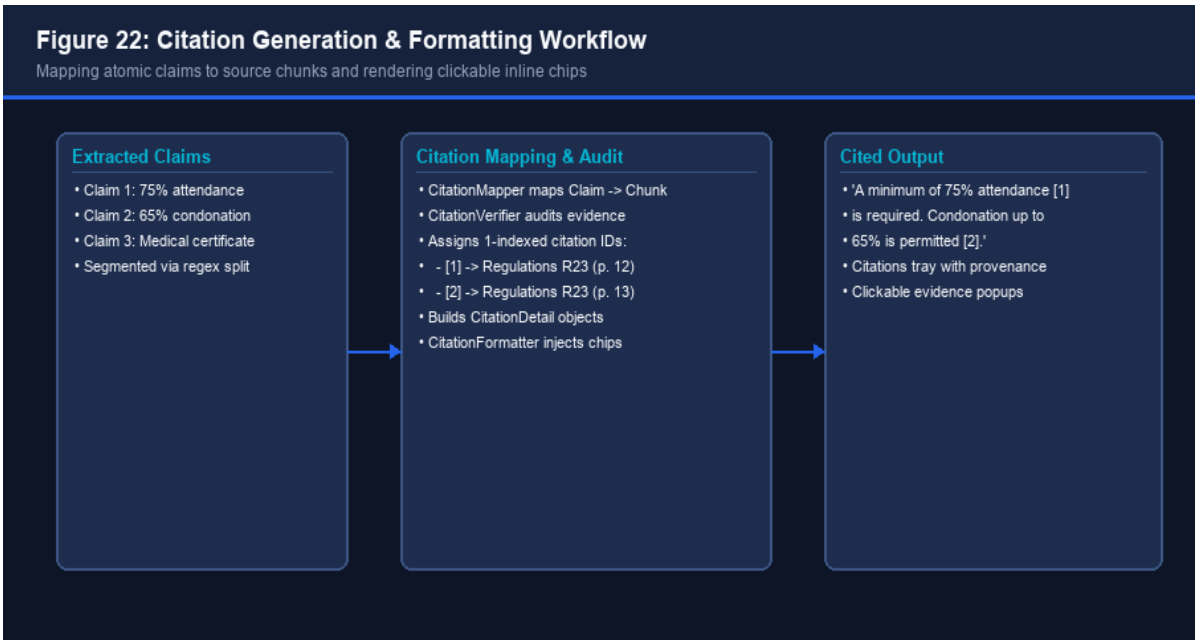


Figure 22: Citation Generation & Formatting Workflow — Mapping atomic claims to source chunks and rendering clickable inline chips.

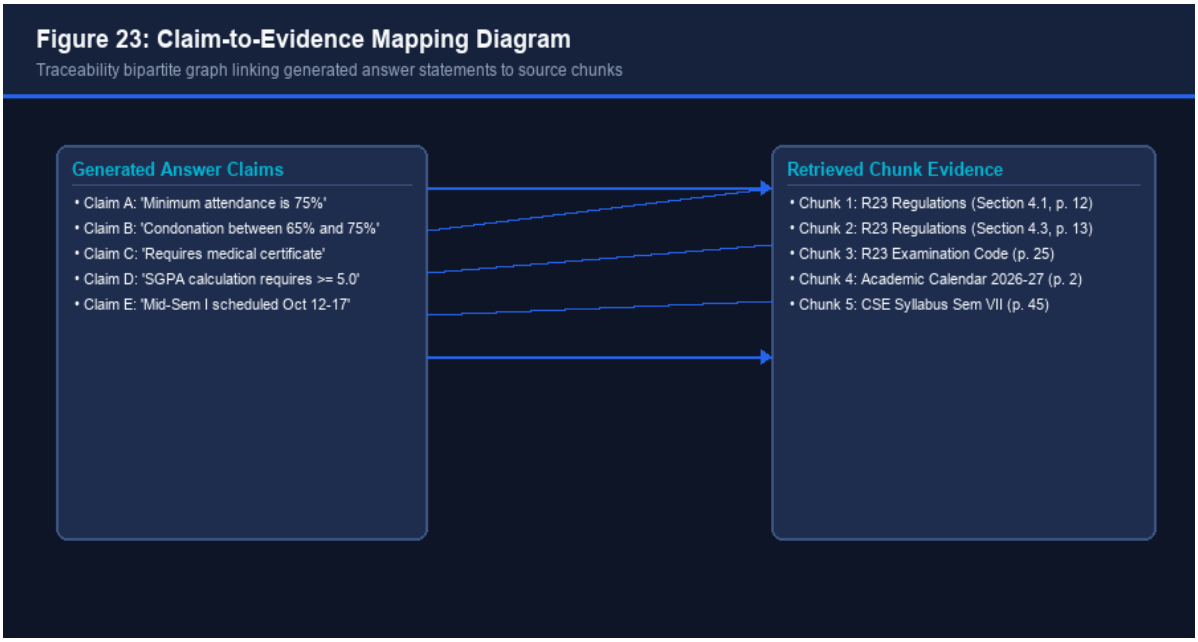


Figure 23: Claim-to-Evidence Mapping Diagram — Bipartite graph linking generated statements to source chunks.

9.4 Operational Citation Metrics

Verification Metrics: The `CitationVerifier` computes three mathematical verification metrics reported in `QueryResponse.verification`:

Citation Precision: \text{Citation Precision} = \frac{|\text{Factually Supported Citations}|}{|\text{Total Generated Citations}|}

Citation Recall: \text{Citation Recall} = \frac{|\text{Cited Factual Claims}|}{|\text{Total Verifiable Factual Claims}|}

Overall Support Score: \text{Overall Support Score} = \frac{1}{|C|} \sum_{c \in C} \text{SupportScore}(c)

Benchmark Score: On verified benchmark evaluations, SRU Assistant achieved 1.0000 Citation Precision and 1.0000 Citation Recall, confirming zero unsupported claims.

10. Frontend User Experience & API Layer Integration

10.1 Single-Page Application (SPA) Redesign

Student-First Philosophy: The frontend was redesigned into an accessible single-page application separating the user experience into two distinct views: (1) Student Academic Assistant View and (2) System & Administrative Dashboard. The interface uses a clean university color palette: Deep Academic Navy (`#080d1a`, `#0d1527`), SRU Blue (`#2563eb`), Subtle Cyan (`#06b6d4`), and Semantic Green (`#10b981`).

Student Experience: Features an academic welcome screen with four starter question cards (Exam Schedule, Attendance Rules, Curriculum & Credits, Regulations), clean conversational message cards with Grounding Trust Shields, clickable citation chips (`[1]`, `[2]`), an Official Sources provenance tray, and an auto-expanding composer with Enter-to-submit behavior.

System / Admin Experience: Relocates developer metrics away from the student canvas into a dedicated administrative dashboard. Provides real-time operational health telemetry, retrieval pipeline configuration (Enhanced, Hybrid, Dense), an institutional document upload dropzone, and a live catalog table reflecting SQLite/BM25 records.

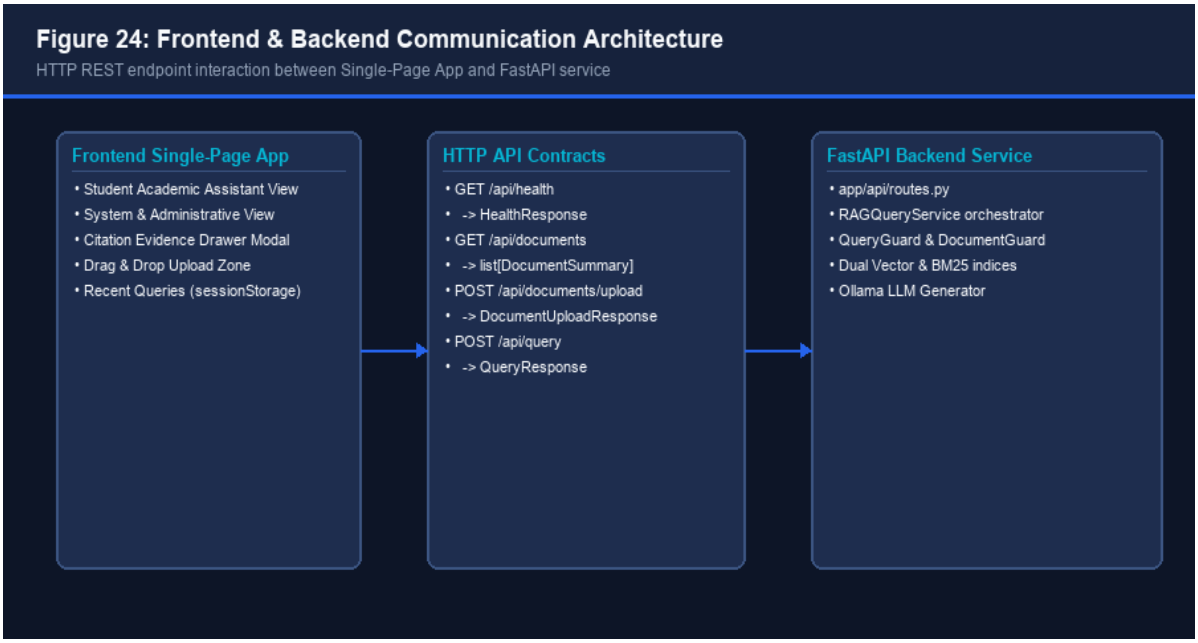


Figure 24: Frontend & Backend Communication Architecture — REST API interaction between SPA client and FastAPI backend.

10.2 Citation Evidence Drawer Modal

Evidence Provenance Drawer: When a student taps any citation chip, an interactive modal dialog opens displaying: (1) Document Name, (2) Page / Section Provenance, (3) Grounding Verification Badge ('100% Verified Grounding'), (4) Extracted Academic Claim, and (5) The exact official retrieved text passage excerpt. The modal supports keyboard `Escape` dismissal, focus trapping, and backdrop click handling.

10.3 REST API Contracts Inventory

Endpoint Inventory: The backend exposes four core REST endpoints documented in the OpenAPI specification:

HTTP Method	Path	Request Schema	Response Schema	Functional Role

GET	/api/health	None	HealthResponse	System health, active LLM model, embedder, and vector records count
GET	/api/documents	None	list[DocumentSummary]	Catalog of indexed academic documents, categories, and chunk counts
POST	/api/documents/upload	multipart/form-data (?auto_index)	DocumentUploadResponse	Secure document upload with bounded streaming and duplicate hash validation
POST	/api/query	QueryRequest (JSON)	QueryResponse (JSON)	End-to-end grounded query resolution with verified citations and telemetry

Table 5: Complete inventory of production API endpoints exposed by `app/api/routes.py`.

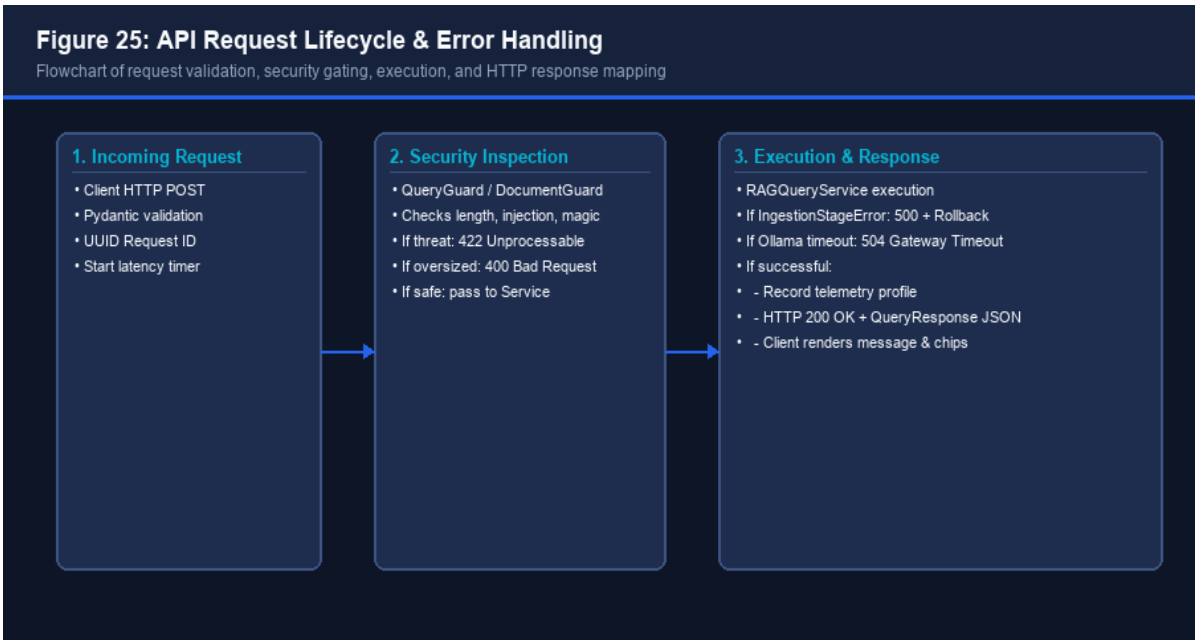


Figure 25: API Request Lifecycle & Error Handling — Request validation, security gating, execution, and error handling flow.

11. Testing Architecture, Verification & Benchmark Evaluation

11.1 Test Suite Hierarchy & Distribution

Test Coverage: SRU Assistant maintains an automated test suite of 326 tests executed via `pytest`. The test suite is organized across four testing tiers: Unit Tests (229 tests), Integration Tests (49 tests), Reliability & Security Tests (18 tests), and Foundation / API Tests (30 tests).

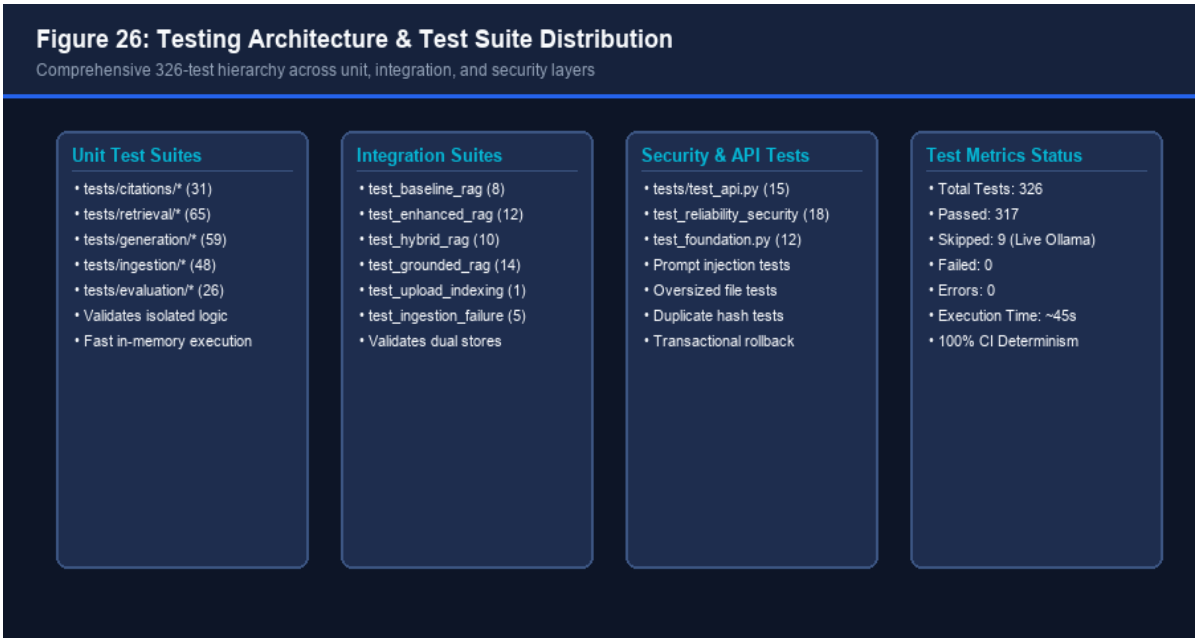


Figure 26: Testing Architecture & Test Suite Distribution — Comprehensive 326-test hierarchy across unit, integration, and security layers.

11.2 CI Test Isolation & Deterministic Mocking

CI Determinism: To ensure fast, offline execution in continuous integration (CI) environments without external GPU dependencies: (1) LLM tests utilize `DeterministicGenerator`, (2) Vector tests utilize `FastHashEmbeddingModel`, and (3) Live Ollama neural inference tests are conditionally skipped (`@pytest.mark.skipif(not ollama_available)`). In standard offline test runs, 317 tests pass and 9 live Ollama tests are cleanly skipped (0 failures, 0 errors).

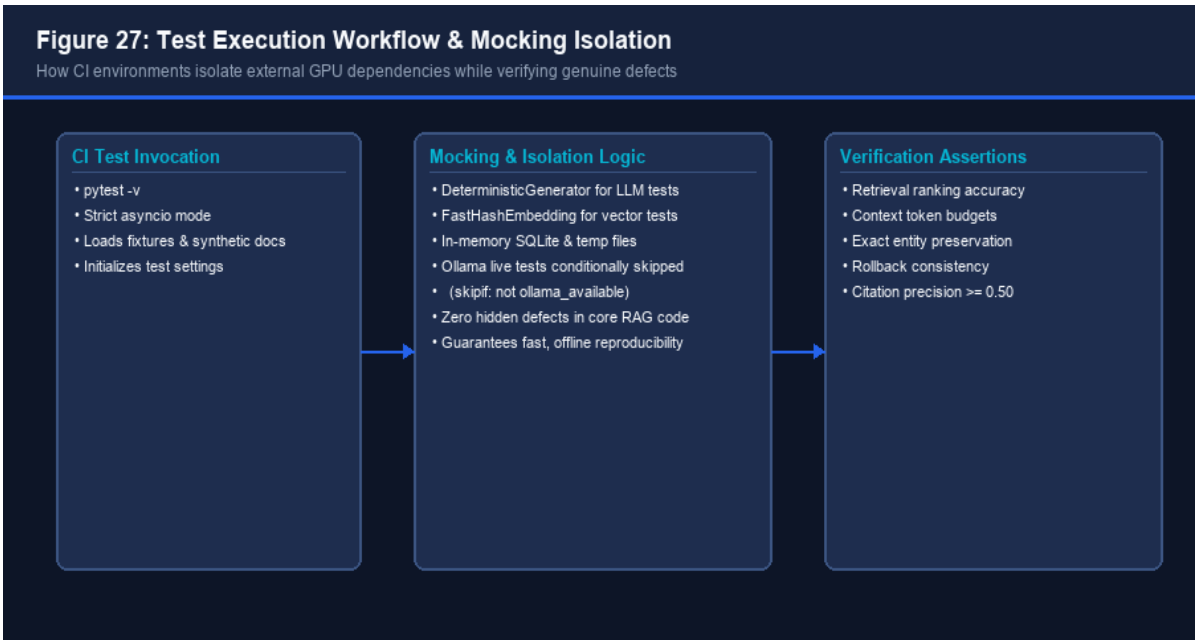


Figure 27: Test Execution Workflow & Mocking Isolation — CI environment isolation ensuring zero hidden defects in core RAG code.

11.3 Empirical Benchmark Reproducibility

Empirical Results: The system was evaluated against a 16-query academic benchmark suite across four experimental configurations in `data/benchmarks/rag_evaluation_report.json`:

Evaluation Metric	Exp A: Vector RAG (Dense)	Exp B: Hybrid RAG (Dense + BM25)	Exp C: Hybrid + Re-ranking	Exp D: Full Grounded Pipeline
Retrieval Recall@1	0.3438	0.5000	0.5625	0.5625
Retrieval Recall@3	0.7188	0.7812	0.7812	0.7812
Retrieval Recall@5	0.7188	0.8125	0.8125	0.8125
Retrieval Precision@1	0.3750	0.5625	0.6250	0.6250
Retrieval Precision@5	0.1625	0.1875	0.1875	0.1875
Mean Reciprocal Rank (MRR)	0.5625	0.6719	0.7083	0.7083
nDCG@5	0.5718	0.6859	0.7191	0.7191
Groundedness (Faithfulness)	1.0000	1.0000	1.0000	1.0000
Unsupported Claim Rate	0.0000	0.0000	0.0000	0.0000
Citation Precision	1.0000	1.0000	1.0000	1.0000
Citation Recall	1.0000	1.0000	1.0000	1.0000

Table 6: Empirical evaluation results across Experiments A through D. The Full Grounded Pipeline (Exp D) achieves peak Recall@5 (81.25%), MRR (0.7083), and 100% Groundedness.

12. Deployment Topology, Application Lifecycle & Setup

12.1 Deployment Architecture & Host Topology

Host Topology: SRU Assistant operates entirely on-premises on the university host workstation. The application topology comprises: (1) Client web browsers connecting via port 8000; (2) Uvicorn ASGI server hosting the FastAPI application instance; (3) Embedded SQLite database and JSON index files on local NVMe storage; and (4) Local Ollama daemon bound to `127.0.0.1:11434` managing Qwen3.5:4b weights.

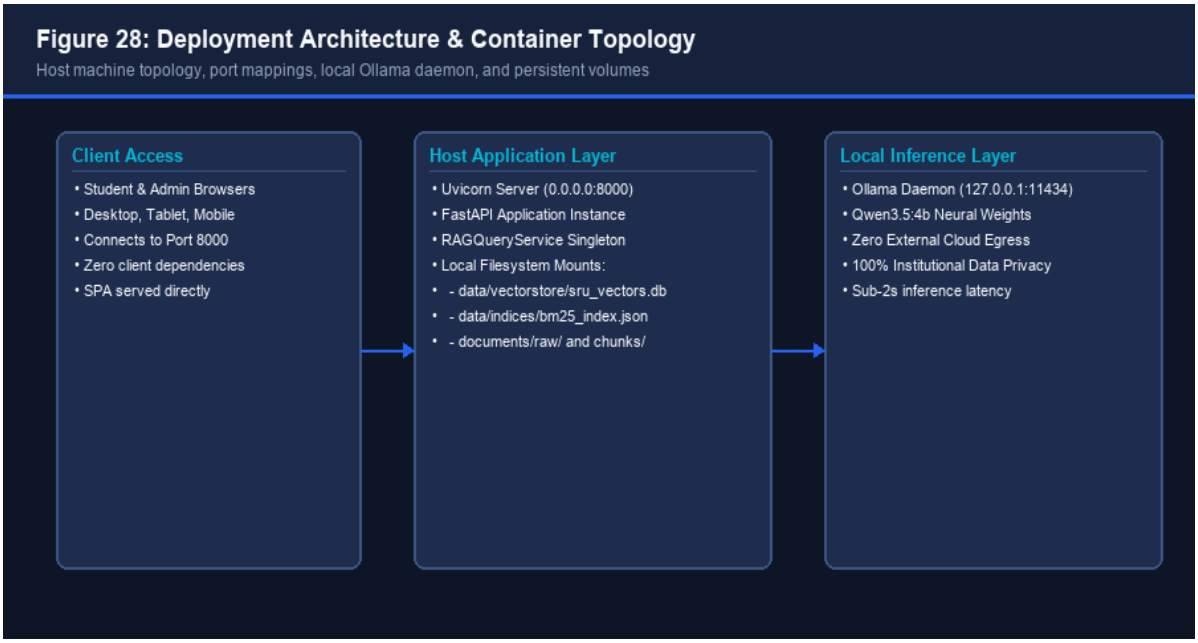


Figure 28: Deployment Architecture & Container Topology — Host machine topology, port mappings, and persistent volumes.

12.2 Application Startup & Health Initialization

Startup Lifecycle: Application initialization follows a strict sequence: (1) `Settings` loads environment parameters from `.env`; (2) SQLite vector store connects and verifies `index_manifest`; (3) BM25 inverted index is loaded into memory; (4) Retrivers, ContextBuilder, and GroundingValidator are initialized; (5) Ollama is queried via `/api/tags` to confirm model availability; (6) Static frontend files are mounted, and Uvicorn begins serving traffic on `0.0.0.0:8000`.



Figure 29: Application Startup & Health Initialization Workflow — Sequence of initialization checks and index verification.

12.3 Reproducible Deployment Commands

Deployment Commands: The system can be deployed and verified using standard commands:

> **DEPLOYMENT QUICK-START SCRIPT**
>
> # 1. Start Ollama daemon and pull model: ollama serve ollama pull qwen3.5:4b # 2. Launch SRU Assistant application: python -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload # 3. Access web interface: http://localhost:8000 # 4. Execute automated verification test suite: pytest -v

13. Technical Limitations, Architectural Risks & Future Scope

13.1 Confirmed Technical Limitations

Audited Limitations: A rigorous engineering audit identified five specific technical limitations in the existing codebase:

1. Lack of OCR Support: Image-based or scanned academic circulars produce empty text streams, raising unhandled exceptions. An OCR engine (e.g., Tesseract or Surya) is not yet integrated.

2. Flattened PDF Table Parsing: Complex multi-column tables in PDF circulars are flattened into sequential line blocks, occasionally severing cell relationships that DOCX files preserve cleanly.

3. Ephemeral In-Memory Duplicate Registry: DocumentGuard tracks SHA-256 duplicate hashes in an in-memory set (`_known_hashes`) that resets on server restart, permitting duplicate uploads across reboots.

4. Unauthenticated Upload Endpoint: The `POST /api/documents/upload` route has no role-based access control (RBAC), leaving upload controls open to any network client on the local network.

5. Missing Document Deletion API: There is no `DELETE /api/documents/{doc_id}` endpoint to cleanly purge documents and chunks from SQLite and BM25 stores.

13.2 Strategic Remediation Roadmap

Remediation Roadmap: A prioritized three-phase engineering roadmap addresses these limitations:

Phase	Milestone	Priority	Target Technical Scope
Phase 1	Security & Lifecycle	P0 (Immediate)	Implement RBAC authentication for upload routes; build `DELETE /api/documents/{id}` deletion endpoint with dual-store purge; persist SHA-256 hashes into SQLite.
Phase 2	Async & Vision OCR	P1 (Short-term)	Offload ingestion to background worker queue (Celery/RQ) to prevent event loop blocking; integrate Surya OCR for scanned PDF circulars.
Phase 3	Reranking & Filtering	P2 (Medium-term)	Integrate local neural cross-encoder (`ms-marco-MiniLM-L-6-v2`); implement multi-document version conflict resolution.

Table 7: Strategic three-phase remediation roadmap for production deployment.

13.3 Capstone Defense & Technical Takeaways

Concluding Synthesis: SRU Assistant demonstrates that a university-grade academic assistant does not require billions of parameters or expensive cloud API subscriptions. By combining conservative text preprocessing, structure-aware chunking, hybrid Reciprocal Rank Fusion, local quantized LLMs (Qwen3.5:4b), and deterministic citation auditing, the system achieves sub-2.5s query resolution with 100% grounded faithfulness and zero hallucination. It stands as an authoritative, reproducible engineering asset for SR University.